

University Student-Support Case Agent - Week 4 Authorization, Failure Handling and Human Approval

Group A

Name	Student Number
Namatovu Christine Maria	2300701098
Opiyo Oscar	2300701330
Isio Lily	2300700445
Babirye Sandra	2300707270
Ogwal Richard	2300716574

BSE4104 Emerging Trends in Software Engineering

Prepared against: Week 4 Tool Catalogue and Tool/API Schemas (Isio Lily), Week 4 Tool/Function-Calling Implementation (Opiyo Oscar, src/tools.py), Project Charter AI Boundary Matrix (Week 1). Author: Ogwal Richard.

1. Authorization checks

Every tool call passes through `guarded_call()` in `src/guardrails.py` before Oscar's `call_tool()` ever runs. The session's `student_id` is the authenticated identity; a call whose `student_id` does not match the session is refused before the tool body executes, and the refusal is written to the guardrail audit log (`evidence/traces/guardrail_audit.jsonl`).

- `get_timetable / create_support_case`: request `student_id` must equal session `student_id`; there is no lookup-on-behalf-of-another-student mode.
- `get_case_status`: additionally, the case record's stored `student_id` must match the session. Not-found and wrong-owner return identically worded refusals so the response does not leak whether a case ID exists for someone else (tested as W07).
- `search_knowledge_base`: any authenticated session may query; the corpus holds no student-specific data, so no ownership check applies.
- Missing `student_id` is reported as a schema problem (`invalid_arguments`), not an auth problem, so the agent asks for the missing field precisely (tested as W06).
- Unknown tool names fail fast with `unknown_tool`: no such capability exists, so even a successful prompt-injection cannot conjure a new action (tested as W14).

2. Failure behaviour

Failures return small structured envelopes that the agent maps to user-safe wording; no failure path fabricates a record or guesses an answer. Verified by `tests/run_week4_tests.py` (14/14 PASS, traces W01-W14).

- `invalid_arguments` (W05, W06, W12, W13): schema violation, including smuggled privileged fields such as `status`; the tool never executes and the agent names the missing field.

- unauthorized (W07, W08): session mismatch or unowned case; refused before execution, logged as a boundary event.
- not_found (W09, W10): unknown ID, or valid ID with no record; agent states it cannot find the record rather than inventing one.
- service_unavailable (W11): missing or malformed store file, or retriever crash; the tool returns the envelope and the agent asks the student to retry later with nothing changed.
- malformed_response: second line of defense in `validate_tool_response()`, which re-checks each tool's output shape (entries list, status enum, results list, draft-only record). A misshapen result stops the turn instead of reaching the student.

3. Unavailable services and unexpected responses

Each tool wraps its store access in `FileNotFoundError` and `JSONDecodeError` handlers, so a deleted or corrupted `data/timetable.json` or `data/cases.json` degrades to `service_unavailable` instead of a stack trace (W11 simulates this by pointing the store at a nonexistent file). Retrieval crashes are caught the same way inside `search_knowledge_base`. Unexpected tool output is caught twice: once by `call_tool()`'s malformed-response guard and again by `validate_tool_response()` in the safety layer.

4. Human approval before higher-impact actions

`create_support_case` is the catalogue's only write action, so it is the only path that requires human approval. `guarded_call()` never calls the write tool directly: it validates arguments first, then records the draft in `data/approvals.json` as `PENDING_APPROVAL` and returns the approval ID (W04). Nothing is written to `data/cases.json` at request time. A staff member completes the action out of band with `approve(approval_id, approver)`, which executes the held write with status still locked to Pending. There is deliberately no auto-approve function anywhere in the codebase.



```

Terminal
user@makerere: ~/projects/guardrails$ python src/guardrails.py --request-approval \
> --student 2300701330 \
> --category missing_marks --subject "..." --description "..."
{
  "approval_id": "APR-0001",
  "state": "PENDING_APPROVAL",
  "student": "2300701330",
  "category": "missing_marks",
  "subject": "...",
  "description": "..."
}

user@makerere: ~/projects/guardrails$ python src/guardrails.py --approve APR-0001 \
> --approver staff@makerere.ac.ug
{
  "state": "APPROVED",
  "case_id": "CASE-1003",
  "status": "Pending"
}

```

The terminal Demonstrated end to end: `python src/guardrails.py --demo` shows a normal lookup, a cross-student refusal, a missing-parameter rejection, a malformed-query rejection,

an unknown-tool rejection, a write held as APR-0001, and its approval into CASE-1003. Demo side effects are reverted afterwards so the seed stores keep only CASE-1001/1002.

5. Updated architecture

docs/architecture/Week4_Tools_Architecture.pdf shows the Week 4 flow: student query enters the Flask orchestrator, which runs parse, AUTHORIZE, APPROVAL GATE, CALL TOOL, and VALIDATE in order; reads reach the Oscar tool registry while the single write is diverted to the human-approval box; stores, the Gemini model, and the deliberately absent approve/grade/admit/fee tools sit in the bottom layer. This diagram supersedes the Week 3 RAG-only diagram for tool-related flows; RAG retrieval now lives inside search_knowledge_base rather than as a separate path.